Web Graphics

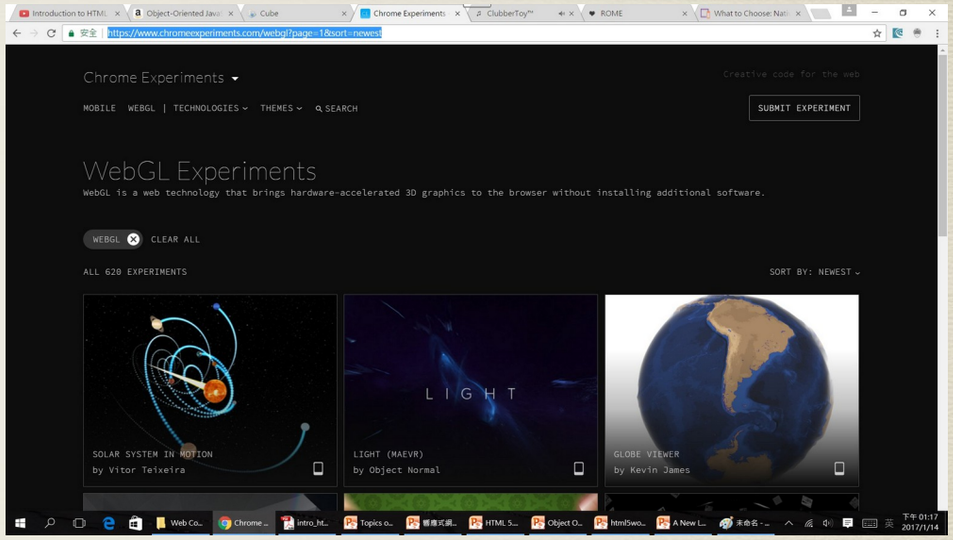
source:https://www.w3schools.com

Graphics on the Web

- 2D Graphics
  - Canvas – The <canvas> element provides API to draw 2D graphics using javascript
  - SVG – Scalable Vector Graphics
- 3D Graphics
  - WebGL — the 3D graphics API for the Web
- CSS 3D transformation & animation

Ball Pool


Hello!
This is how it works:
1. Drag a ball
2. Click on the background
3. Shake your browser
4. Double click
5. Play!

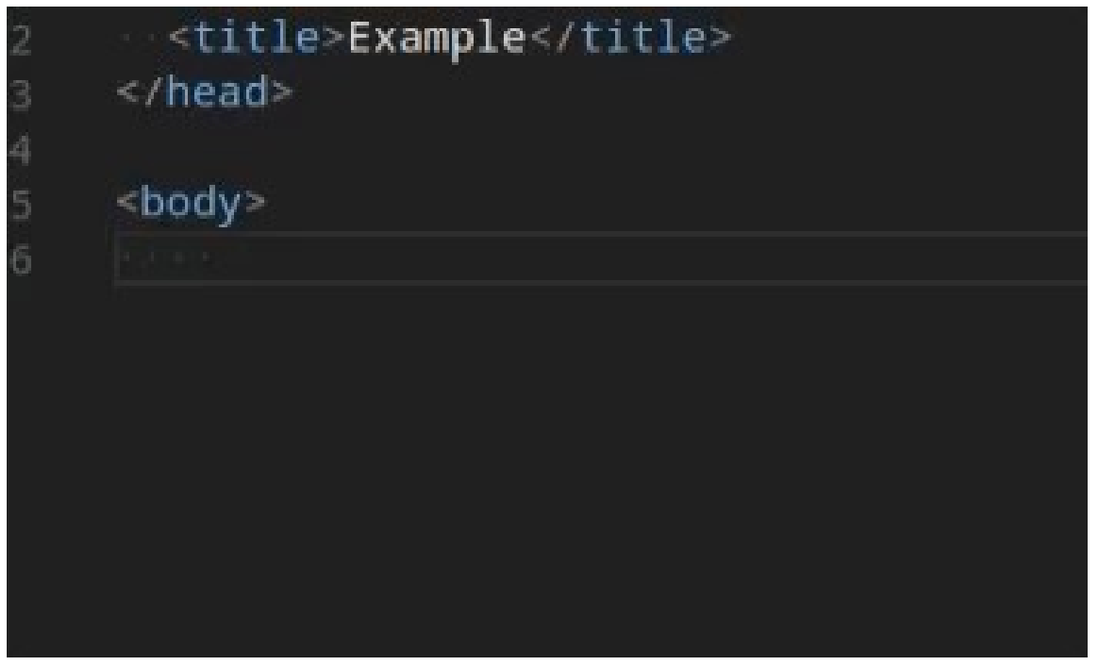

Chrome Experiments
MOBILE WEBGL | TECHNOLOGIES THEMES SEARCH
SUBMIT EXPERIMENT
WebGL Experiments
WebGL is a web technology that brings hardware-accelerated 3D graphics to the browser without installing additional software.
WEBGL CLEAR ALL
ALL 620 EXPERIMENTS
SORT BY: NEWEST
SOLAR SYSTEM IN MOTION
by Vitor Teixeira
LIGHT
LIGHT (MAEVR)
by Object Normal
GLOBE VIEWER
by Kevin James

HTML 5 Components

* HTML (tag)
* CSS (style)
* JavaScript (behavior)

HTML5 editor in this course

- Visual Studio Code - Code Editing. Redefined
  - https://code.visualstudio.com/



HTML 5 Template

```
<!DOCTYPE  html>
<html>
   <head>
     <meta     charset="UTF-8">
     <title>Title of the document</title>
     <style>     </style>
     <script> </script>
   </head>
   <body>

     Content of the document……
     <script> </script>

   </body>
</html>
```

The <!DOCTYPE html> declaration defines that this document is an HTML5 document
The <html> element is the root element of an HTML page
The <head> element contains meta information about the HTML page
The <title> element specifies a title for the HTML page (which is shown in the browser's title bar or in the page's tab)
The <body> element defines the document's body, and is a container for all the visible contents, such as headings, paragraphs, images, hyperlinks, tables, lists, etc.
The <style> tag is used to define style information (CSS) for a document. It must be included inside the <head> section of the document.
The <script> tag is used to embed a client-side script (JavaScript).

Popular HTML Tag

Tag

- …
- <h1></h1>…..<h6></h6>
- <p>
- <img src="path">
- link
-
  - item1
  - item2
  - item3
-

The element is used as a container for other HTML elements.

HTML headings are defined with the <h1> to <h6> tags.
HTML paragraphs are defined with the <p> tag.

HTML images are defined with the <img> tag. The source file (src), alternative text (alt), width, and height are provided as attributes.

HTML links are defined with the <a> tag. The link's destination is specified in the href attribute.

An unordered list starts with the <ul> tag. Each list item starts with the <li> tag.

CSS

3

□CSS (cascading style sheet, 階層式樣式表)

□CSS is a language that describes the style of an HTML document.

□CSS describes how HTML elements should be displayed.

□CSS3 is the latest standard for CSS.

CSS 3

- Selector – 更有彈性的選擇器
- Webfonts – 嵌入式字體
- Layout – 多樣化的排版選擇
- Styling radius gradient shadow – 圓角、漸層、陰影
- Border background – 邊框的背景支援
- Transition – 元件的移動效果
- Transform – 元件的變形效果
- Animation – 將移動和變形加入動畫支援

CSS 語法 -Internal CSS

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: linen;
}

h1 {
  color: maroon;
}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

CSS element Selector

div a container for HTML elements

Selecto
r

Property:
value

```
div    {   background-color:
           aqua; color:#FF0000;
           border: 1px solid
           red; width:100px;
           height:100px;
           text-
           align:center;
}
```

CSS id Selector

Selecto
r

Property:
value

```
#id    {   background-color:
           aqua; color:#FF0000;
           border: 1px solid
           red; width:100px;
           height:100px;
           text-
           align:center;
}
```

```
<p id="para1">Hello World!</p>
```

CSS class Selector

Selector

Property:

value

```
.center {
        background-color:
        aqua; color:#FF0000;
        border: 1px solid
        red; width:100px;
        height:100px;
        text-
        align:center;
}
```

```
<p class="center">Hello World!</p>
```

CSS 語法 -Inline CSS

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;text-
align:center;">This is a heading</h1>
<p style="color:red;">This is a
paragraph.</p>

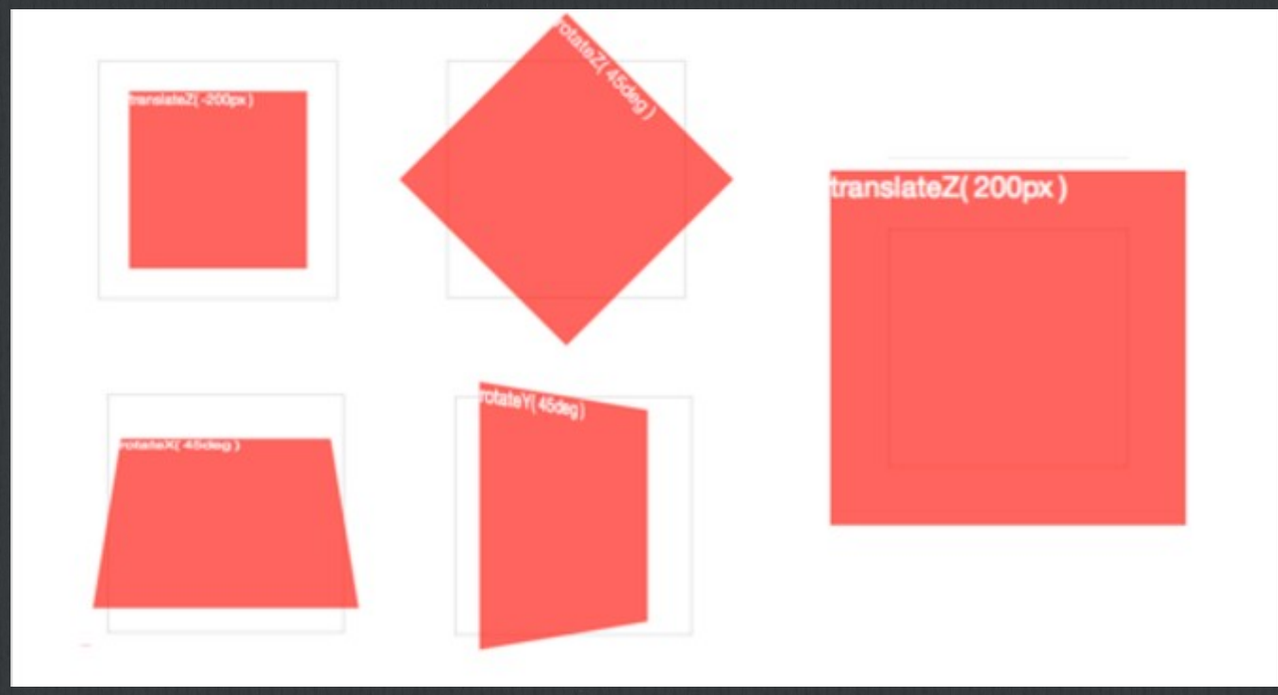
</body>
</html>
```

CSS3 Transformation

- rotateX(angle)
- rotateY(angle
-)
- rotateZ(angle
-)
- translateZ(tz)
- scaleZ(sz)
- translate3d(tx, ty, tz)

  scale3d(sx, sy, sz)

  rotate3d(rx, ry, rz, angle)



CSS3 3D Transformation



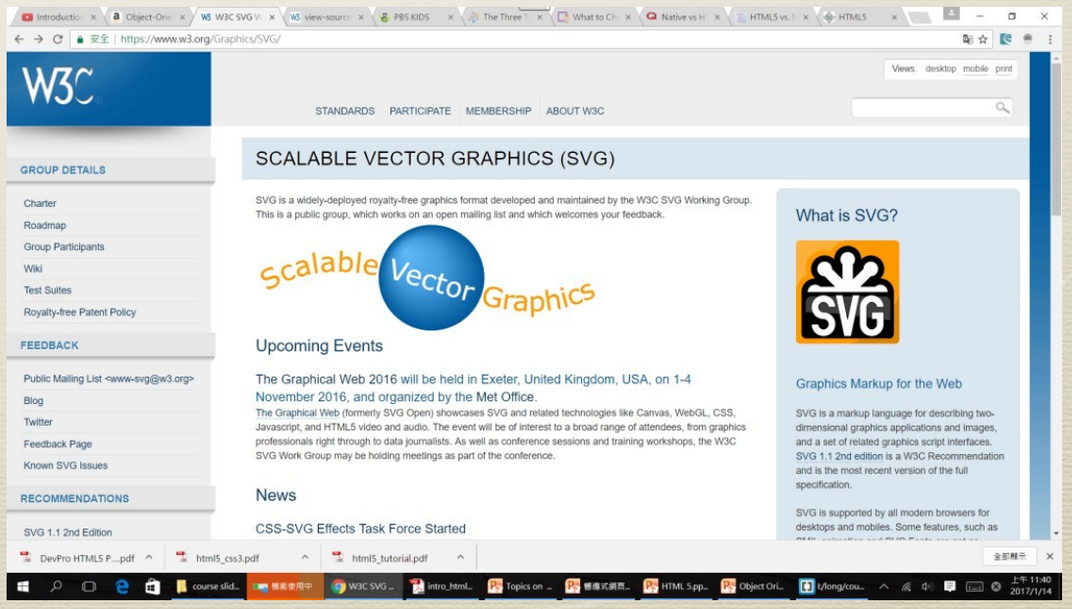


SVG (Scalable Vector Graphics)

*Graphics Markup for the Web

*SVG is an excellent way to create interactive, resolution-independent vector graphics that will look great on any size screen.





```
<!DOCTYPE html>
<html>
 <head>
   <script>
     function doCircle(evt)          {
    var theCircle = evt.target;
    var radius = theCircle.getAttribute("r");
     if (radius == 50)
            radius =
    75; else radius =
    50;
        theCircle.setA
        ttribute("r",ra
        dius);
    }
 </script>
</head>
<body>
 <svg
 width="300p
 x"
 height="300p
 x">
   <text x="25"
   y="50" font-
```

SVG animation

```
<!DOCTYPE html>
<html>
<body>

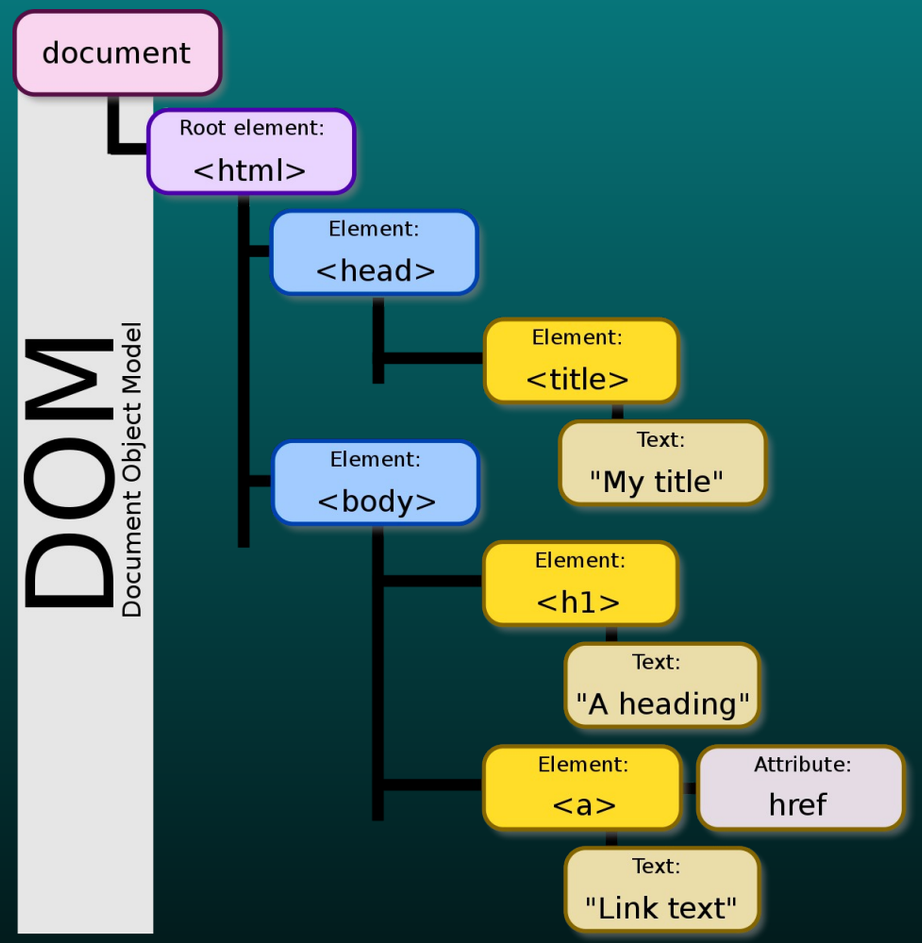
<h2>SVG animate Element</h2>

<svg width="100%" height="100" xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="50" style="fill:red;">
    <animate
      attributeName="cx"
      begin="0s"
      dur="10s"
      from="50"
      to="90%"
      repeatCount="indefinite" />
  </circle>
  Sorry, your browser does not support inline SVG.
</svg>

</body>
</html>
```

The HTML DOM (Document Object Model) —wiki

- When a web page is loaded, it is created as a DOM Tree of Object by the browser.
  With the HTML DOM, JavaScript can access and change all the elements of an HTML document.

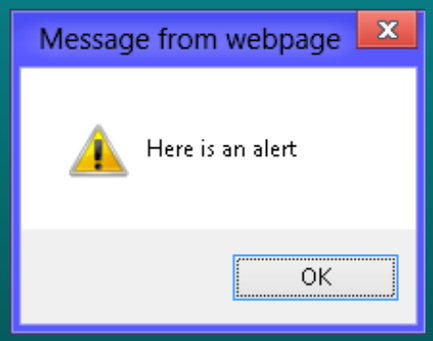


Accessing DOM objects

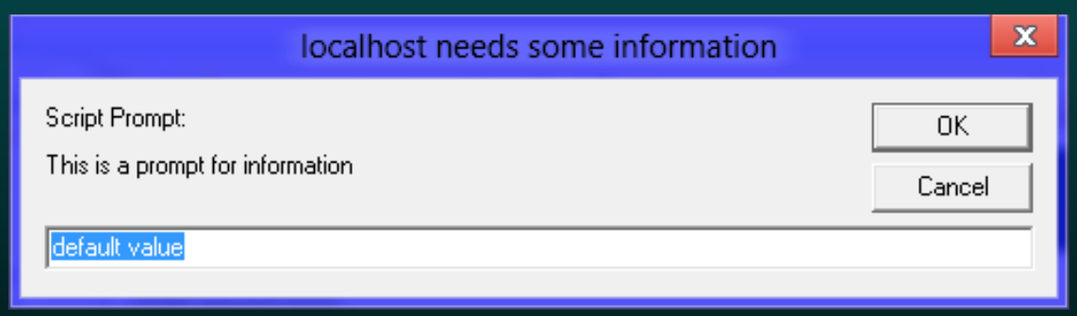
- getElementById Returns a reference to the first object with specified id
  - var btn = document.getElementById('btnSave');
- getElementsByTagName Returns a live NodeList 陣列
  - var images = document.getElementsByTagName('img');

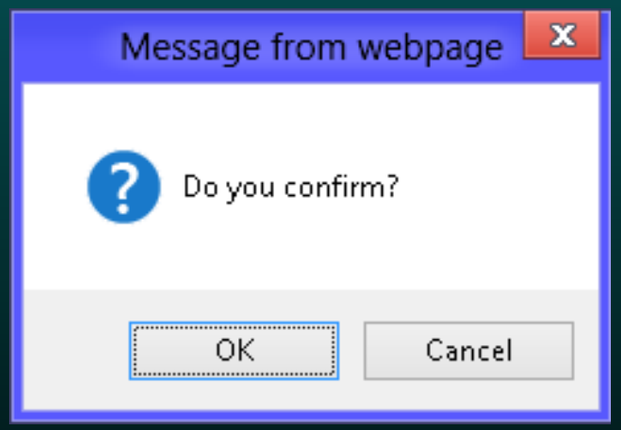
JavaScript Popup Boxes

- alert("Here is an alert");



- var promptResult = prompt("This is a prompt for information", "default value");





- var promptResult = confirm("Do you confirm?");

HTML5 Canvas

HTML Canvas

- Draw Text
- Draw 2D/3D Graphics
- Animated
- Move
- respond to JavaScript events and user action (key clicks, mouse clicks, button clicks, finger movement).
- Used in Games

HTML Canvas

- <canvas id="myCanvas" width="200" height="100" style="border:1px solid #000000;">
  </canvas>

- <script>
  - var c = document.getElementById("myCanvas");
  - var ctx = c.getContext("2d");

- </
  script>

<canvas> element is used to draw graphics on a web page.

border: border-width border-style border-color;
border - CSS：层叠样式表 | MDN

The getContext() method returns an object with tools (methods) for drawing.

Draw Text: font, fillStyle

```
fillText(text, x, y , maxWidth)
```

```
strokeText(text, x, y , maxWidth)
```

maxWidth(Optional) The maximum text width in pixels

```
<canvas id="myCanvas" width="500" height="300"></canvas>
<script>
    var canvas = document.getElementById("myCanvas");
    var context = canvas.getContext("2d");
    context.font = "40pt Ariel";
    context.fillStyle = "blue";
    context.fillText("Hello World!", 150, 100);
</script>
```

Draw a rectangle

`fillRect(x, y, width, height)`
Draws a filled rectangle.

`strokeRect(x, y, width, height)`
Draws a rectangular outline.

`clearRect(x, y, width, height)`
Clears the specified rectangular area, making it fully transparent.



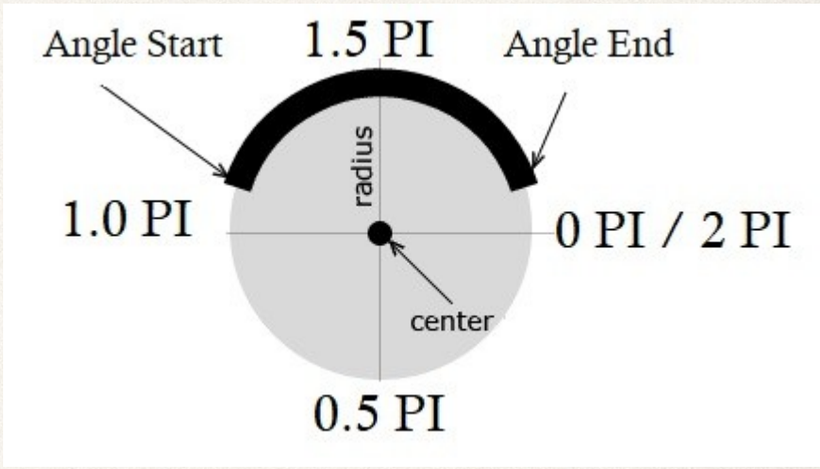
```
const canvas =
document.getElementById("myCanvas");
const ctx = canvas.getContext("2d");

ctx.fillStyle="#FF0000";
ctx.fillRect(20, 20, 200, 100);
```

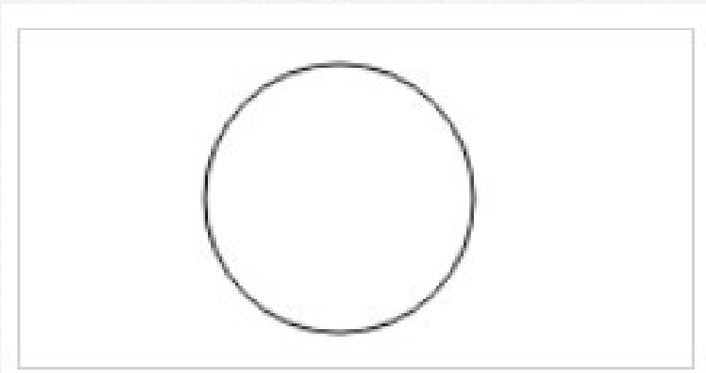
context.fillStyle = *color*|*gradient*|*pattern*

Draw a Circle

- beginPath() - begins a path
- arc(x,y,r,startangle,endangle)




```
<script>
    var c = document.getElementById("myCanvas");
    var ctx = c.getContext("2d");
    ctx.beginPath();
    ctx.arc(95,50,40,0,2*Math.PI);
    ctx.stroke();
</script>
```



Don't forget

use the stroke() method to actually draw

Draw a Line: moveTo(), lineTo()

- var canvas = document.getElementById("myCanvas"); var ctx = canvas.getContext("2d");
  ctx.beginPath();
- ctx.moveTo(20, 10);
- ctx.lineTo(150, 80)
- ctx.lineWidth = "10";
- ctx.strokeStyle = "green";
- ctx.lineCap = "round";
- ctx.stroke();

Draw a Path



* A path is a list of points, connected by segments of lines that can be of different shapes, curved or not, of different width and of different color.

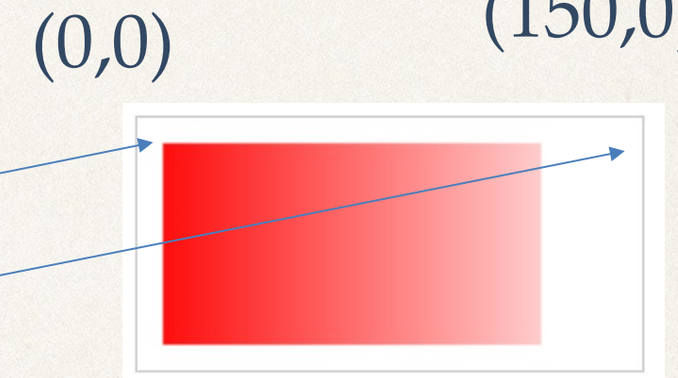
false is default

context.arc(*x, y, r, sAngle, eAngle, counterclockwise*)

```
ctx.beginPath();
ctx.arc(75, 75, 50, 0, Math.PI * 2, true); // Outer circle
ctx.moveTo(110, 75);
ctx.arc(75, 75, 35, 0, Math.PI, false); // Mouth (clockwise)
ctx.moveTo(65, 65);
ctx.arc(60, 65, 5, 0, Math.PI * 2, true);  // Left eye
ctx.moveTo(95, 65);
ctx.arc(90, 65, 5, 0, Math.PI * 2, true);  // Right eye
ctx.stroke();
```

Canvas - Gradients

- var c=document.getElementById("myCanvas");
  var ctx=c.getContext("2d");
  / / Create gradient
  var grd=ctx.createLinearGradient(0,0,150,0);
  grd.addColorStop(0,"red");
  grd.addColorStop(1,"white");
  / / Fill with gradient
  ctx.fillStyle=grd;
  ctx.fillRect(10,10,150,80);



context.createLinearGradient(*x0*, *y0*, *x1*, *y1*)

Parameter Values

Param	Description
x0	The x-coordinate of the start point of the gradient
y0	The y-coordinate of the start point of the gradient
x1	The x-coordinate of the end point of the gradient
y1	The y-coordinate of the end point of the gradient



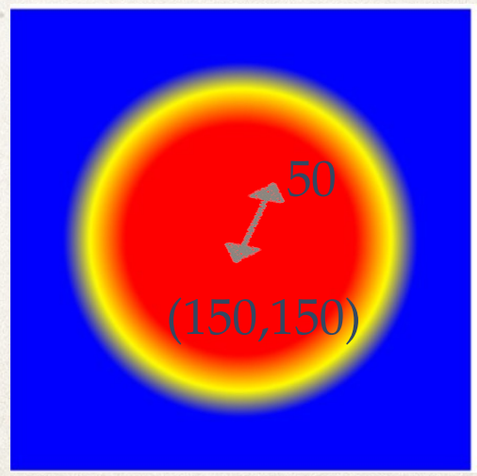
Radial Gradier

context.createRadialGradient(*x0, y0, r0, x1, y1, r1*)

Parameter Values

Parameter	Description
x0	The x-coordinate of the starting circle of the gradient
y0	The y-coordinate of the starting circle of the gradient
r0	The radius of the starting circle
x1	The x-coordinate of the ending circle of the gradient
y1	The y-coordinate of the ending circle of the gradient
r1	The radius of the ending circle

```
<!DOCTYPE html>
<html>
   <head> <meta charset="utf-8"></head>
   <body>
      <canvas id ="a_canvas"  width="300"  height="300">
      </canvas>
      <script>
         var canvas = document.getElementById ("a_canvas");
         var ctx = canvas.getContext("2d");
         var g = ctx.createRadialGradient(150,150,50,150,150,100);
            g.addColorStop(0,"red");
            g.addColorStop(0.5,"yellow");
            g.addColorStop(1.0,"blue");
            ctx.fillStyle = g;
            ctx.fillRect(20,20,260,260);
      </script>
   </body>
</
html>
```



Load image

```
<!DOCTYPE html>
<html>
<body>

<img id="logo" width="220" height="277" src="logo.jpg" alt="logo.jpg">

<canvas id="myCanvas" width="240" height="297" style="border:1px solid
grey;"></canvas>

<script>
const canvas = document.getElementById("myCanvas");
const ctx = canvas.getContext("2d");
const img = document.getElementById("logo");
ctx.drawImage(img, 10, 10);
</script>

</body>
</html>
```

drawImage(

Position the image on the canvas:

context.drawImage(*img*, *x*, *y*)

Position the image on the canvas, and specify width and height of the image:

context.drawImage(*img*, *x*, *y*, *width*, *height*)

Clip the image and position the clipped part on the canvas:

context.drawImage(*img*, *sx*, *sy*, *swidth*, *sheight*, *x*, *y*, *width*, *height*)

Param	Description
img	Specifies the image, canvas, or video element to use
sx	Optional. The x coordinate where to start clipping
sy	Optional. The y coordinate where to start clipping
swidth	Optional. The width of the clipped image
sheight	Optional. The height of the clipped image
x	The x coordinate where to place the image on the canvas
y	The y coordinate where to place the image on the canvas
width	Optional. The width of the image to use (stretch or reduce the image)
height	Optional. The height of the image to use (stretch

Lab Time

Try to draw any interesting graph using canvas API